

About

This lab allows you to practice writing Boolean expressions, **if-elif-else** statements, and transforming control flow diagrams into code.

Learning Objectives

1. Practice writing boolean expressions and **if-elif-else** statements
2. Practice understanding and transforming control flow diagrams into code.

Estimated Size

1 program file.

Setup

In your Python files, you must include author information. On the first line, add a **# Author comment line**, where you mention your full name. Similarly, include **your email and Spire ID** on the **# Email** and **# Spire ID** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678
```

0. Lists and Conditions

You will be implementing two functions in this assignment. Both of these functions manipulate lists of integers. You may use the following starter code and name it **list_class.py**:

```
# AUTHOR    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678

# Below, please implement the combine_lists() function as specified in the
# pre-lab instructions.
# ----- YOUR CODE STARTS HERE -----

# ===== YOUR CODE ENDS HERE =====

# You can use the code below to help test your functions

# Uncomment the following lines (remove the # characters on each line)
# to test combine_lists()
```

```
#print(combine_lists([1, 2, 3],[4, 5, 6, 7]))
#print(combine_lists([1, 2, 3, 4, 5],[4, 5, 6, 7]))

# Uncomment the following lines (remove the # characters on each line)
# to test classify_by_length()
#print(classify_by_length([1, 2, 3]))
#print(classify_by_length([1, 2, 2, 3]))
#print(classify_by_length([]))
```

1. Implement `combine_lists` (10 points)

The first task is to implement a function called `combine_lists`. This function has two parameters: a list object with integers, and another list object with integers. You are asked to do the following tasks:

- Take the two lists `a` and `b` as parameters.
- Insert the first element of `b` at the beginning of `a`.
- Insert the last element of `b` at the end of `a`.
- Delete the middle element of `a`. You may assume that `a` had an odd number of elements before the deletion.
- Return the resultant list `a` after all the changes.

The starter code contains example code that you can uncomment to help **test your function**. Follow the instructions there. Below are some examples of running a correct implementation of this function:

```
print(combine_lists([1, 2, 3],[4, 5, 6, 7]))
[4, 1, 3, 7]
print(combine_lists([1, 2, 3, 4, 5],[4, 5, 6, 7]))
[4, 1, 2, 4, 5, 7]
```

2. Implement `classify_by_length` (10 points)

Your task is to write a function, called `classify_by_length`, which should:

- Take one list of integers, `a` as the parameter.
- Return a **string** representing the list class. The string must be one of `empty`, `odd_length`, or `even_length`, and it may NOT be anything else.
 - You should return `empty` if the list is empty.
 - You should return `odd_length` if the list has an odd length.
 - You should return `even_length` if the list has an even length.

The starter code contains example code that you can uncomment to help **test your function**. Follow the instructions there. Below are some examples of running a correct implementation of this function:

```
print(classify_by_length([1, 2, 3]))
odd_length
print(classify_by_length([1, 2, 2, 3]))
```

```
even_length
print(classify_by_length([]))
empty
```

3. Submit your code to Gradescope

Log in to [Gradescope](#), select **Extra Credit Practice Lab 03**, and submit **list_class.py**. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline. Only one person per group should submit.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure, it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions, which will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the [UMass Academic Honesty Policy and Procedures](#) so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty, you should re-read the course policies. We assume you have read the course policies in detail, and by submitting this project, you have provided your virtual signature in agreement with these policies.